Presentation script

Predicting Machine Failure from Process Parameters · Group 11 · 7:00 minutes · 3 speakers + 1 at the console

	Who	Slides	Time	Name
	Speaker 1	1–2	0:00 – 2:20	_____
	Speaker 2	3–4	2:20 – 4:40	_____
	Speaker 3	5–7	4:40 – 7:00	_____
	Console	all	throughout	_____

How to use this script

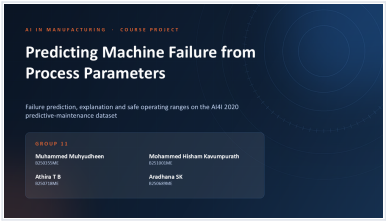
• Each speaker has about **2 minutes 20 seconds**, roughly 290 spoken words at a calm pace (about 125 words per minute).
• Words in *[orange brackets]* are actions, not speech: point at the chart, or say a teammate's name.
• Don't read the slide aloud. The slide shows the numbers; you explain what they mean.
• Practise once with a phone timer. If you run long, drop a sentence, not the handoff line.
• Every technical word is said once with its plain meaning. If anyone asks something deeper (how the model was tuned or tested), say: *"[Console person] will show you that."*
• The technical words each speaker uses, and their one-line meanings, are on the **Words to know** page (page 5).

The story in one breath

Machines give clues before they break (Speaker 1) → we found 10,000 runs, avoided two traps, and built the model step by step (Speaker 2) → it catches 9 in 10 failures and tells a factory exactly how to run safely (Speaker 3).

Speaker 1

Slides 1–2 · 2:20 · about 273 words
Name: _____

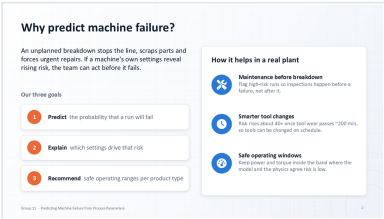


Slide 1: Title

about 20 seconds · 40 words

Good morning, sir, and everyone. We are Group 11. I'm *[your name]*, and with me are *[names]*.

Our project is called "Predicting Machine Failure from Process Parameters". In simple words: can a machine's own readings warn us before it breaks down?



Slide 2: Why predict machine failure?

about 120 seconds · 233 words

Imagine a factory machine that suddenly breaks in the middle of a shift. The production line stops, the parts it was making are wasted, and the repair has to happen in a hurry. That is expensive.

But a machine gives us clues all the time: how fast it spins, how much twisting force it uses, how hot it runs, and how worn its cutting tool is. Our idea was to use these clues to spot danger early. This idea is called predictive maintenance: fixing a machine just before it fails, instead of after it breaks, or on a fixed schedule that may be far too early.

We set three goals. First, predict: for every run of the machine, estimate the chance that it will fail. Second, explain: show which settings push that risk up. Third, recommend: give safe settings for each product type, so the operator knows how to run the machine safely.

[point right] This is why it matters in a real plant. One, maintenance can happen before a breakdown, not after it. Two, smarter tool changes: we found the risk jumps about forty times once a tool has been used for around two hundred minutes, so tool changes can be planned. Three, safe operating windows: a range of power and force where the risk stays low.

To do all this we needed data, and *[Speaker 2]* will now show you what our data looks like.

Speaker 2

Slides 3–4 · 2:20 · about 298 words
Name: _____



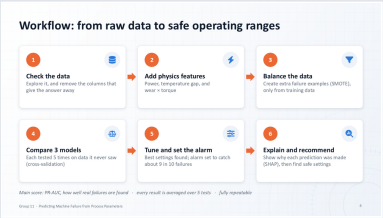
Slide 3: The dataset

about 70 seconds · 157 words

Thank you. We used a public dataset called AI4I 2020. It records ten thousand runs of a milling machine. For each run we know six things: the product's quality type, two temperatures, the speed, the torque, which is the twisting force, and how worn the tool is. And we know whether it failed.

Here is the catch: only 339 runs failed. That is just 3.4 percent, about one failure for every 28 healthy runs. *[point to chart]* The chart shows the five types of failure.

This creates two traps. *[point to cards]* First, five columns record which type of failure happened. They give the answer away, like reading the answer key during an exam, so we removed them. This mistake is called data leakage. Second, because failures are rare, a lazy model that always says "no failure" is 96.6 percent accurate, and still useless. So instead of accuracy, we use a score called PR-AUC, which measures how well the real failures are found.



Slide 4: Workflow

about 70 seconds · 141 words

Here is how we built the solution, in six steps.

One, we checked the data and removed the columns that give the answer away. Two, we added three features from physics: power, the temperature gap, and tool wear times torque, because these are what actually break machines.

Three, failures are rare, so we created extra failure examples for the model to learn from, with a method called SMOTE, and only from the training data, never from the test data.

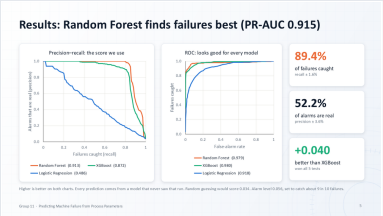
Four, we compared three models. Each was tested five times on data it had never seen. This is called cross-validation, and it makes the results trustworthy.

Five, we fine-tuned the best model and set its alarm so it catches about nine out of ten failures. Six, we explained its decisions and turned them into safe settings.

[Speaker 3] will now show you the results.

Speaker 3

Slides 5–7 · 2:20 · about 284 words
Name: _____



Slide 5: Results

about 70 seconds · 154 words

Thanks. The winner was the Random Forest, with a score of 0.915 out of 1. For comparison, random guessing would score only 0.034.

[point to left chart] This chart shows the key trade-off. Moving right means catching more failures; staying high means fewer false alarms. The orange line, our Random Forest, stays at the top the longest, so it is the best. The score we use is simply the area under each line: the bigger, the better. The blue line, the simplest model, drops quickly, because machine failures don't follow straight-line rules.

[point to middle chart] This second chart is a popular way to compare models, but notice that all three look good on it. That is exactly why we did not rely on it.

[point to cards] In practice, our model catches 89 percent of failures before they happen, and about half of its alarms are real. The other half are cheap extra inspections, a fair price for catching nine out of ten breakdowns.



Slide 6: Summary

about 60 seconds · 113 words

To sum up, four points. Our predictions are reliable. Adding features from physics made a big difference. Our testing was honest: a careless setup could have shown a fake, near-perfect score of 0.999. And the model can explain itself: its reasons match the machine's real failure rules, even though we never told it those rules.

[point right] For a factory, this becomes simple advice: keep power between 4.75 and 8 kilowatts, change the tool before 200 minutes of use, keep torque below 54 newton-metres, and when the temperature gap gets small, keep the speed above 1,390 rpm.

We also built an interactive dashboard, where anyone can move these settings and instantly see the risk change.

Slide 7: Thank you about 10 seconds

Thank you for listening. We're happy to take your questions, and *[console person]* can show you the live dashboard.

Words to know

Each technical word is said once in the talk, with its plain meaning. If someone asks, this is the one-line answer.

Speaker 1

Process parameters	The machine's settings and readings: speed, torque, temperatures, tool wear.
Predictive maintenance	Fixing a machine just before it would fail, instead of after, or on a fixed schedule.
Torque	The twisting force the machine applies.

Speaker 2

Data leakage	When the inputs secretly contain the answer, so the score looks great but means nothing.
PR-AUC	Our main score: how well the model ranks real failures above healthy runs. 1 is perfect, guessing gives 0.034.
SMOTE	Creates extra, realistic failure examples so the model has more to learn from (training data only).
Cross-validation	Testing a model 5 separate times, each time on data it never saw, then averaging.
Random Forest / XGBoost	Models made of many decision trees that vote or correct each other.

Speaker 3

Recall	Out of all real failures, how many we caught (89%).
Precision	Out of all alarms we raised, how many were real failures (52%).
ROC curve	A popular comparison chart that looks good even for weak models when failures are rare.
SHAP	A method that shows how much each setting pushed a prediction up or down.
Operating window	The range of settings where the predicted failure risk stays below 1%.

Console: showing sir the evidence

While the speakers present, the console person sits beside sir with the dashboard and code open, ready to back every claim with the real result.

When	Have this ready on screen
Slide 2	Dashboard open on the What-if simulator tab, default settings: “Within budget, 0.04%”. Ready to show a risky setting if asked.
Slide 3	notebooks/01_eda.ipynb: the class-balance figure and the leakage check (“any flag” gives 97.3% recall without a model).
Slide 4	src/pipeline.py: the SMOTE-in-fold comment; src/cv.py: the PR-AUC comment; src/data.py: the leakage drop.
Slide 5	Dashboard, Model performance tab: drag the threshold slider from 0.5 to 0.056 and show missed failures fall from 62 to 34.
Slide 6	Dashboard, Operating ranges tab: raise tool wear past 200 min and watch the band turn red.
Questions	reports/metrics.md for the full numbers; PROJECT_GUIDE.pdf section 11 for prepared answers.

Before you start

• Start the dashboard ten minutes early: **streamlit run app.py**. The first load takes about 15 seconds.
• Open tabs: the dashboard, *reports/metrics.md*, *notebooks/02_modeling.ipynb* and *PROJECT_GUIDE.pdf*.
• Technical questions (leakage, SMOTE inside folds, PR-AUC, nested cross-validation, threshold choice) come to you. Section 11 of PROJECT_GUIDE.pdf has a short prepared answer for each.

Numbers everyone should know

10,000 runs	339 failures (3.4%)	PR-AUC 0.915 ± 0.006
89% of failures caught	52% of alarms real	guessing scores 0.034
power 4.75–8 kW	change tool before 200 min	fake score if done wrong: 0.999